Applied High Performance Computing

2026-II

José Fiestas PhD

16 de septiembre de 2026

Universidad de Ingeniería y Tecnología
jfiestas@utec.edu.pe



Unidad 4: Comunicación y coordinación

Objetivos:

1. Pasos de Mensaje: MPI, Mensajes Punto a Punto, MPI, Comunicación Colectiva, Blocking vs non-blocking
2. Memoria Compartida: OMP, Constructores y cláusulas
3. Programacion Híbrida

Temática

MPI+OpenMP

Ley de Moore

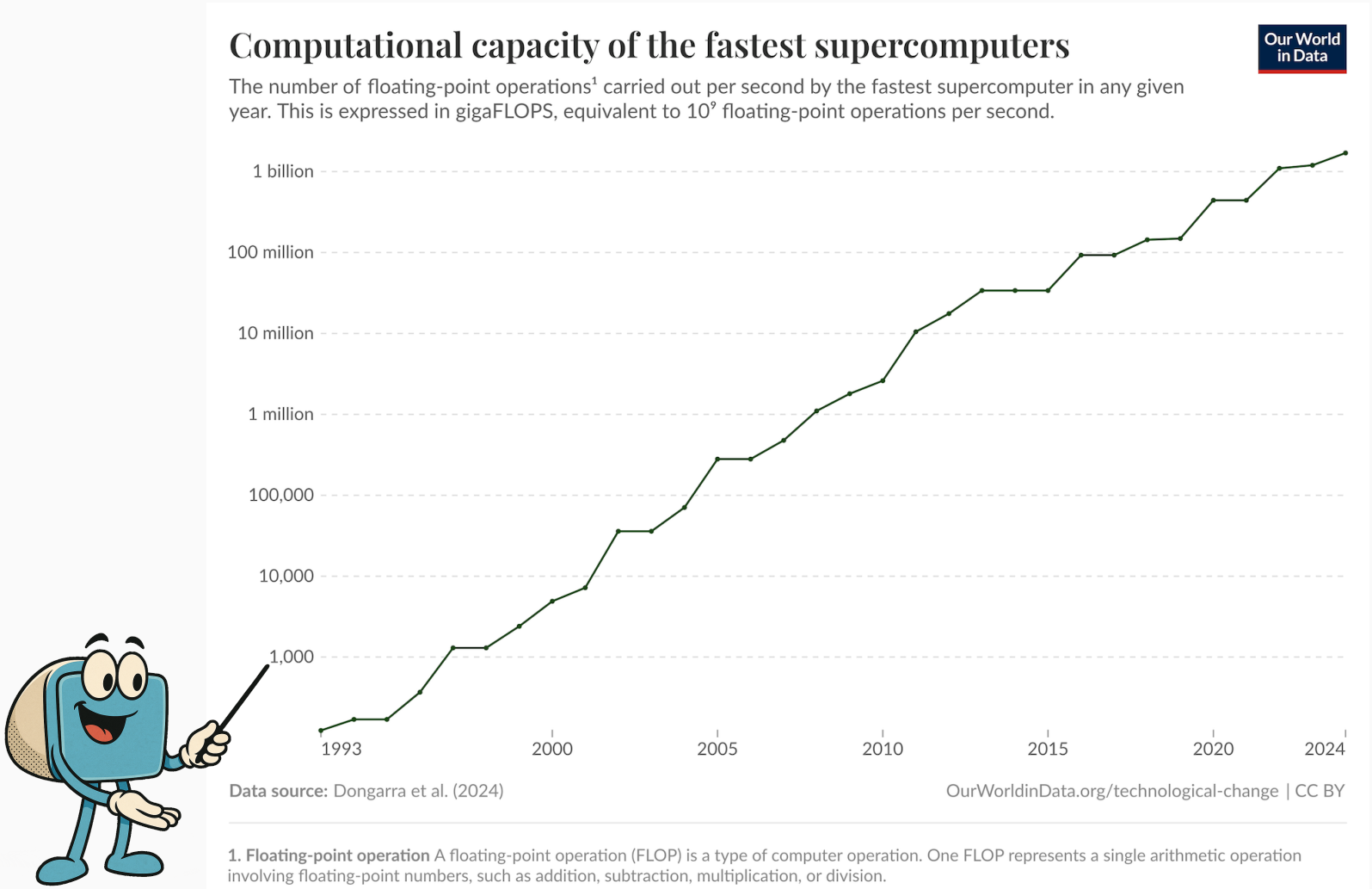
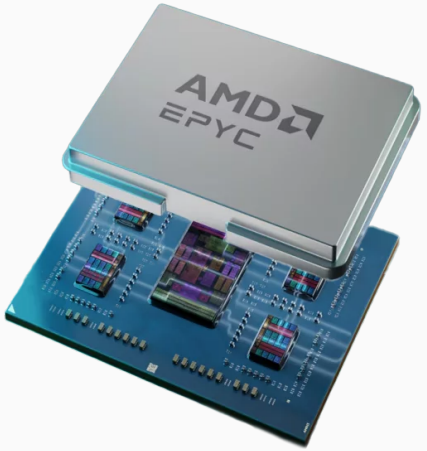


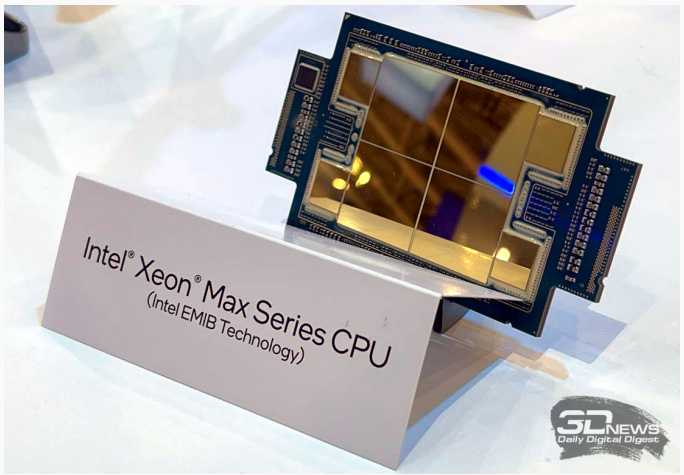
Figura 1: Dongarra et al. (2024) – https://ourworldindata.org/grapher/supercomputer-power-flops

a) **Intel Xeon**
CPU Max 9470: 52 núcleos, 104 hilos, 2 sockets por nodo,

b) **AMD EPYC 9004 Series:**
hasta 96 núcleos, y 192 hilos por socket, hasta 2 sockets por nodo,

c) **Intel Xeon Scalable ("Sapphire Rapids"):** hasta 60 núcleos, 120 hilos, 8 sockets por nodo







Limitaciones: cómputo vs. acceso a memoria

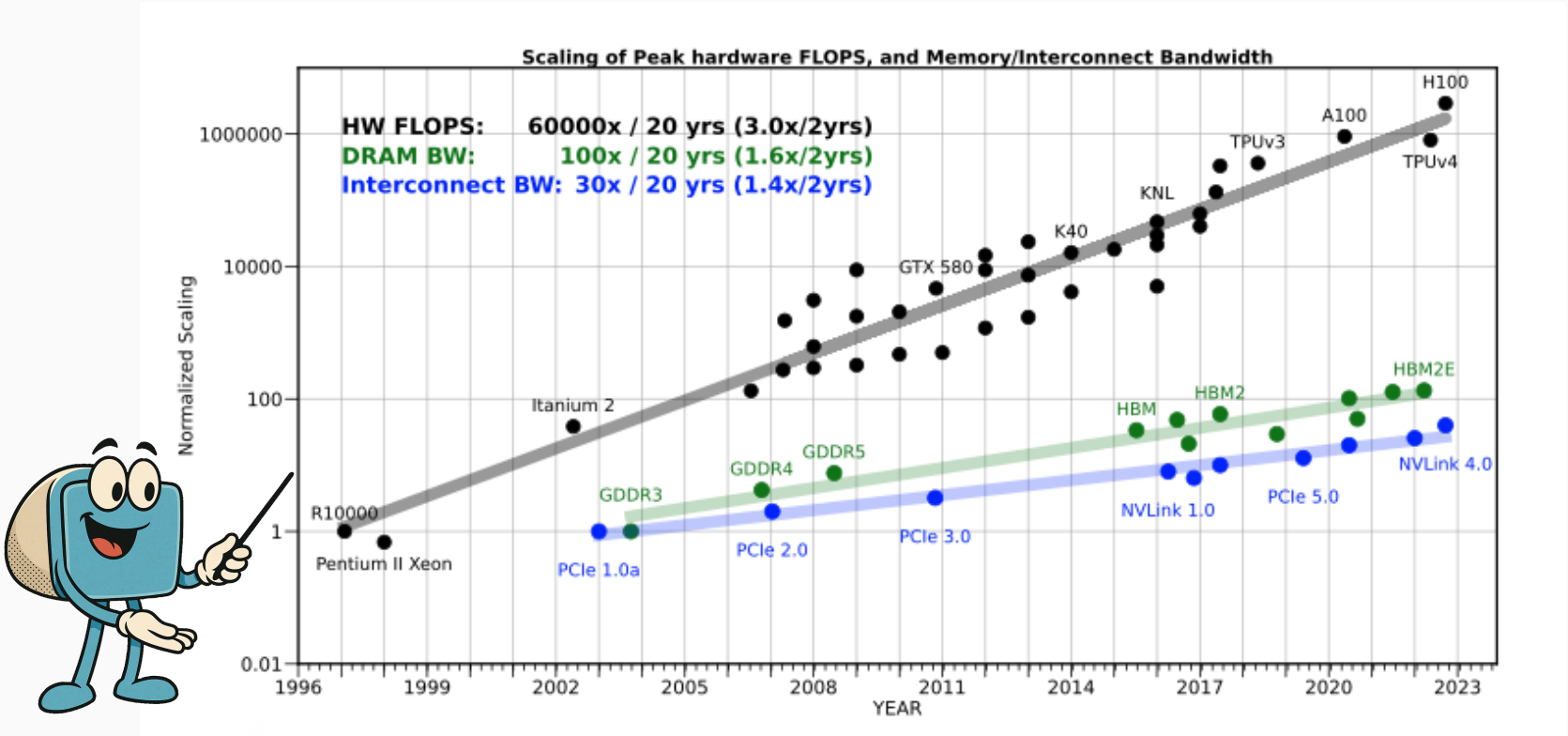


Figura 2: Escalabilidad de ancho de banda en distintas redes de interconexión y memoria, así como de FLOPs. Datos normalizados. De: Gholami et al. 2024 https://arxiv.org/abs/2403.14123

Limitaciones: consumo de energía

La potencia dinámica creciente en la operación del CMOS (Complementary Metal-Oxide-Semiconductor) se define como

$$P_{dyn}(f) = \gamma \cdot f^3$$

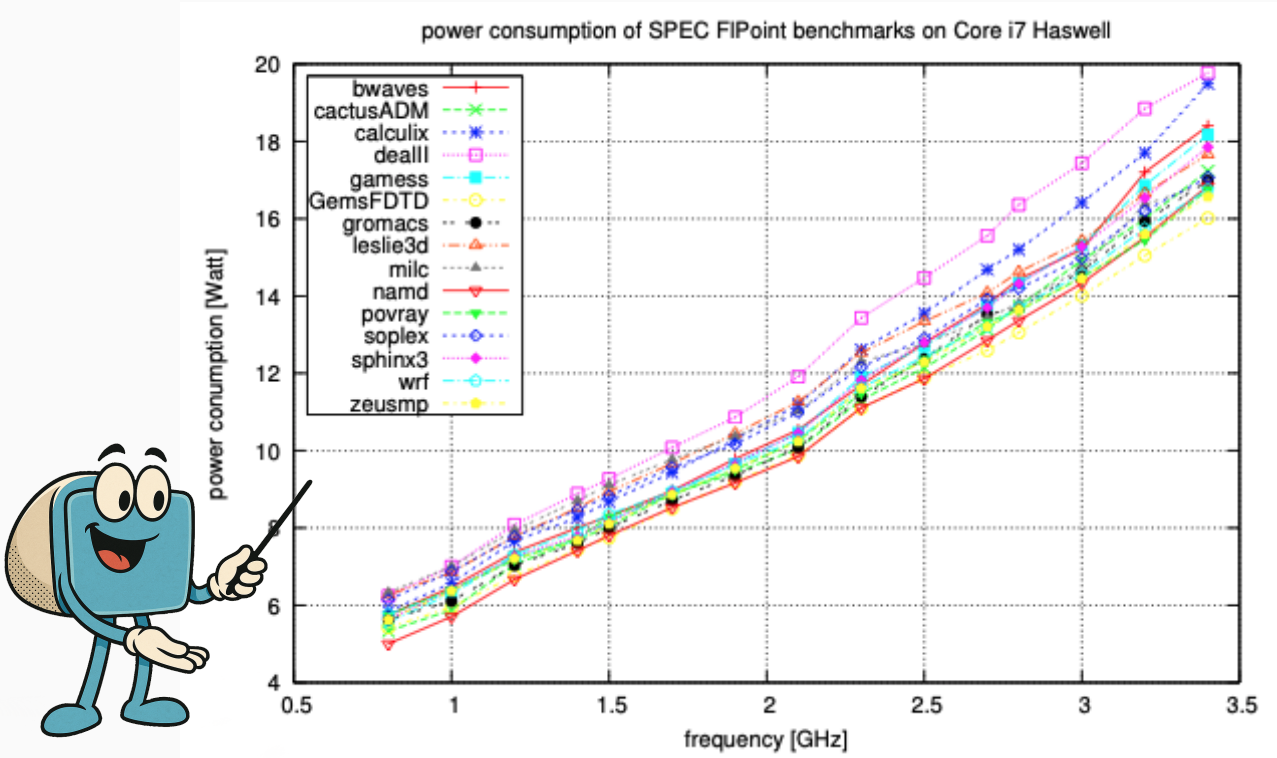


Figura 3: SPEC CPU2006 floating-point benchmarks de consumo de energía en un procesador Intel Core i7 Haswell a distintas frecuencias. De: Rauber et al. 2014 https://link.springer.com/article/10.1007/s11227-014-1236-4

Solución: paralelismo

Por consecuencia, se debe:

1. **Explotar** un número **grande** de núcleos "lentos" (" *The chickens and the ox")*
2. *Aprovechar la* **memoria interna** *(local) de procesadores (paralelismo híbrido)*
3. *Lograr* **niveles altos** *de paralelismo (arquitecturas modernas)*

Consecuencias para desarrolladores de software:

1. Mayor necesidad de comprender la **arquitectura** de hardware
2. Desarrollar códigos se vuelve **más complejo** (trabajo en equipo)

Paradigmas de programación:

1. MPI aún predomina y se mantendrá por buen tiempo (tanto en desarrollo como en aplicaciones)
2. El híbrido MPI-OPENMP-CUDA está siendo mas usado, sobre todo en supercomputadores
3. Nuevos lenguajes de programación paralela en la categoría PGAS (Partitioned Global Address Space) como GASPI, Coarray-Fortran, UPC, Chapel (Cray)

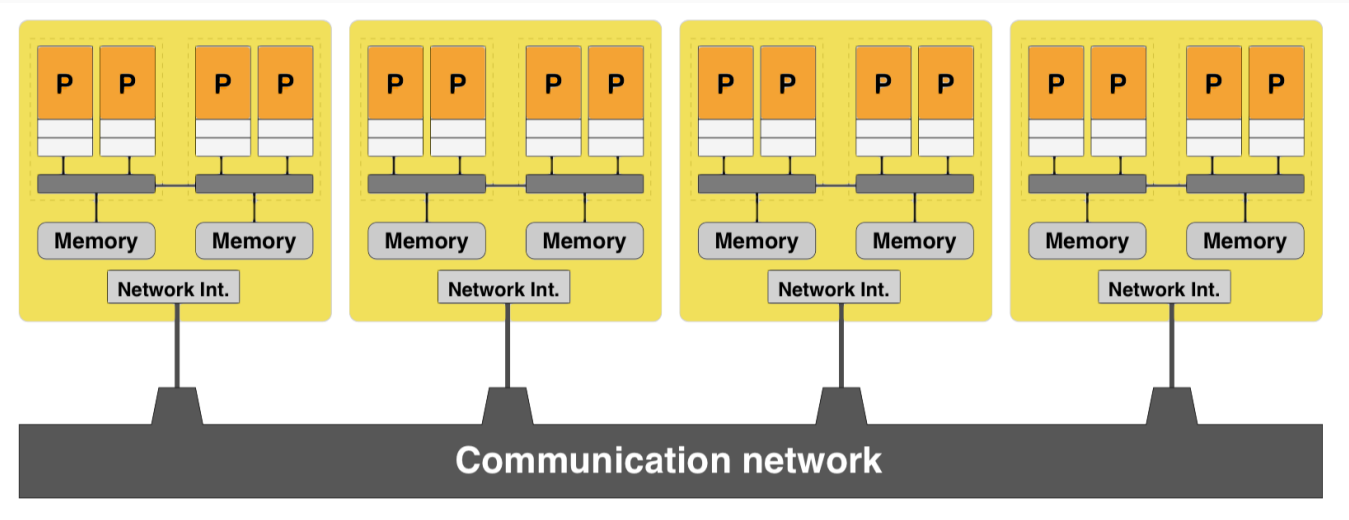


Figura 4: MPI vs OpenMP

Paradigmas de programación:

Memoria compartida

- **Hilos** escriben y leen del RAM
- Se sincronizan de acuerdo a competencia entre tareas
- hilos pertenecen a un proceso

Memoria distribuída

- **Procesos** envían y reciben mensajes
- Solo utilizan variables locales
- poseen uno o más hilos

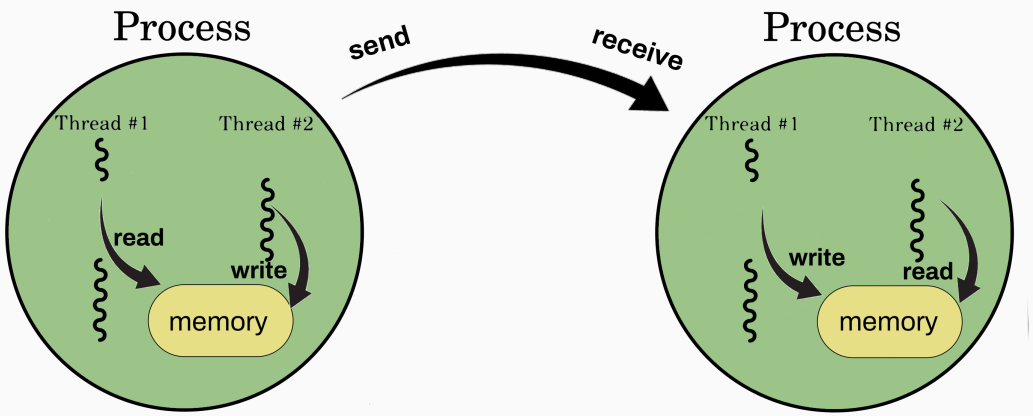


Figura 5: sincronización y comunicación

Paralelismo híbrido

Consiste en la mezcla de paradigmas en paralelismo para explotar las ventajas de cada una de ellas. Es decir, MPI se usará para comunicación entre procesos, y OpenMP dentro de cada proceso

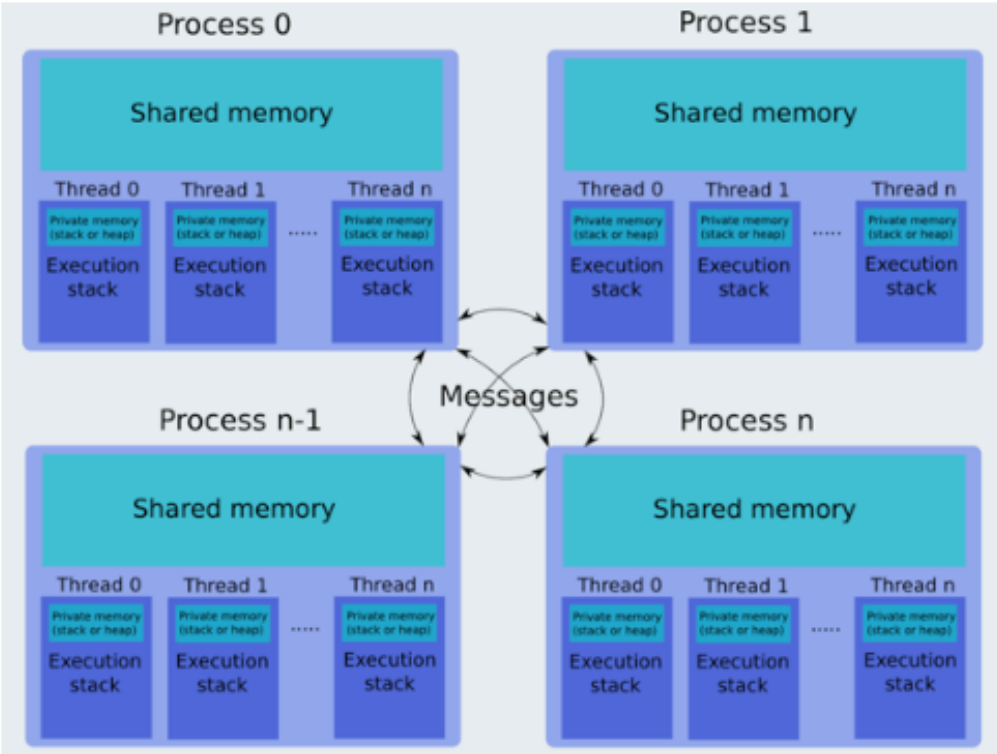


Figura 6: Comunicación

Paralelismo híbrido

Ventajas:

1. Mayor **escalabilidad**. Aumenta **granularidad**
2. Mejor **balance de cargas**
3. Optimiza el **uso de memoria**

Desventajas:

1. Mayor **complejidad** de código
2. Rendimiento en ambos, MPI y OpenMP, debe ser bueno.
3. **Efectos NUMA**

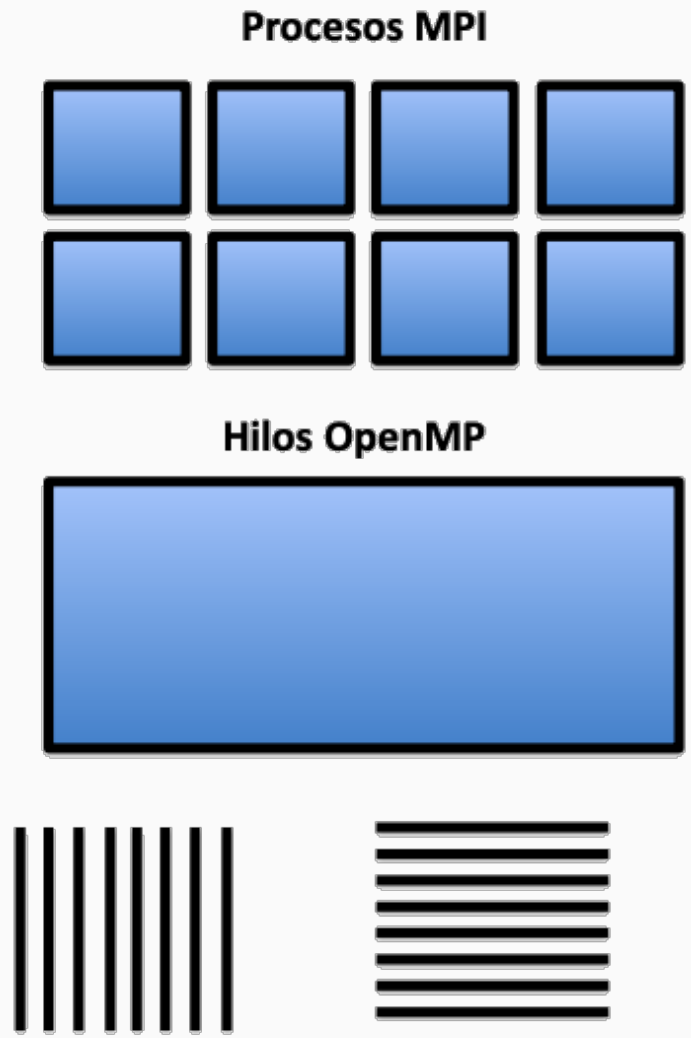
Paralelismo híbrido

Se benefician algoritmos :

1. con **limitada escalabilidad** MPI
2. que requieren **balance de cargas dinámico**
3. que usan hardware con espacio de **memoria limitada** y **alta comunicación** MPI
4. con aplicaciones de **paralelismo masivo**

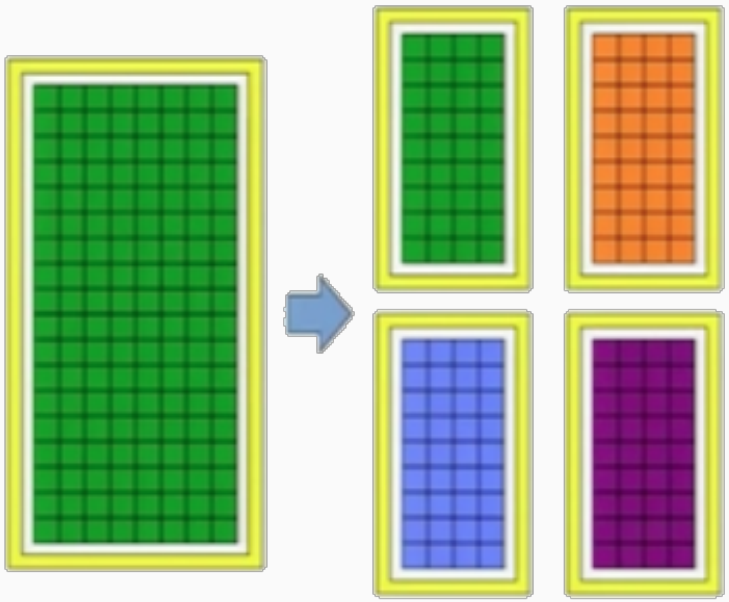
Optimización en el uso de memoria

Regiones de halo son copias locales de información remota que es necesaria para procesamiento de datos. Deben ser copiadas frecuentemente. Utilizando OpenMP se reduce el tamaño de las copias de información de los halos que deben ser almacenadas. Dependiendo de la estructura de datos, se puede aprovechar de la combinación MPI/OpenMP para reducir requerimiento de memoria. Reduciendo regiones de halo también reduce requerimientos de comunicación



Descomposición de dominio

Malla en dos dimensiones:
de 1 a 4 procesos conlleva
a que cada proceso tenga:
- Un cuarto del
número de celdas que computar
- La mitad del número de
celdas en halos para comunicar.
P procesos, cada uno con MxN
celdas y 2M+2N celdas en halo
4P procesos, cada uno con
(M/2)x(N/2) celdas y (M+N)
celdas en halo.

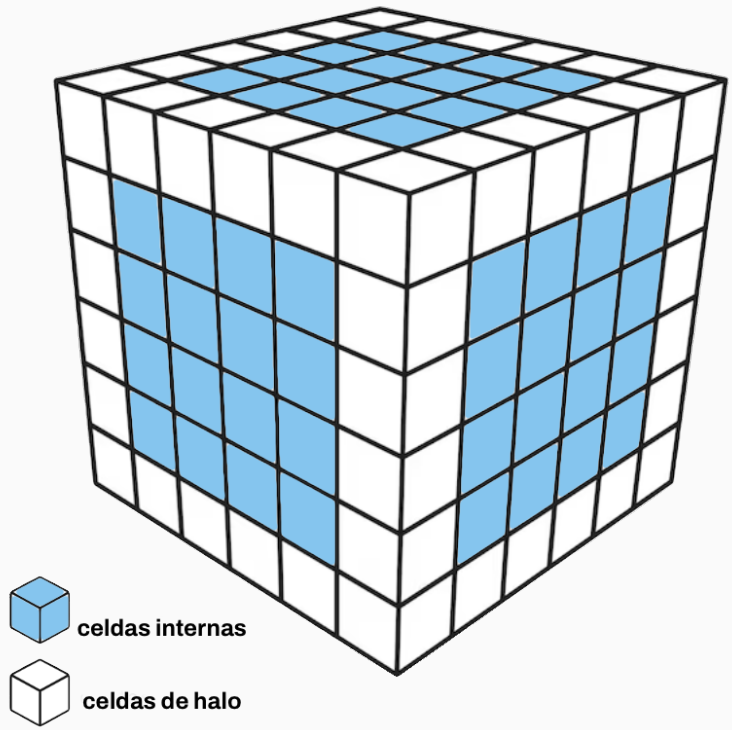


Mientras la cantidad de procesos se incrementa en O(P), la región de halo por proceso disminuye más lentamente, orden $O(\sqrt{P})$.

Descomposición de dominio

Porcentaje de halo por proceso:

En 3D, la data por proceso en un cubo de 10^3 se transformaría en un cubo de 12^3 (728 celdas o 42 % halo), o 20^3 en un cubo de 22^3 (2648 celdas o 25 % halo), o 40^3 elementos con 1 elemento de halo, en un cubo de 42^3 elementos (10088 celdas o 14 % halo)



Descomposición de dominio

E.g. la descomposición de dominio en un modelo PIC (Particle in Cell) para simulación de plasmas. Código Smilei (Deroullat et.al. 2017)

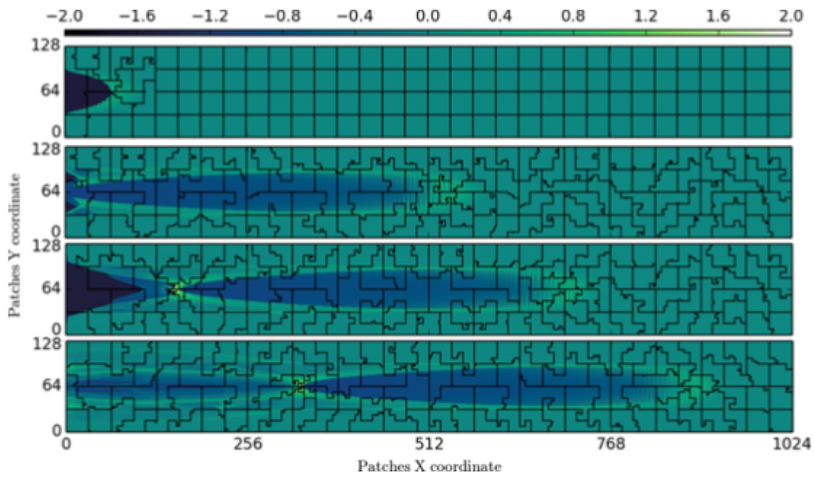


Figure 6: Evolution of the MPI domains shapes with respect to the computational load distribution. The colormap indicates the local imbalance $I_{loc} = \log_{10}(L_{loc}/L_{av})$ where L_{loc} is the local patch computational load and L_{av} the average computational load. Black lines delimit the different MPI domains. The laser enters an initially homogeneous plasma from the left side of the box and propagates towards the right. The 4 panels show the entire simulation domain after 1600, 5480, 6820, 9080 iterations from top to bottom.

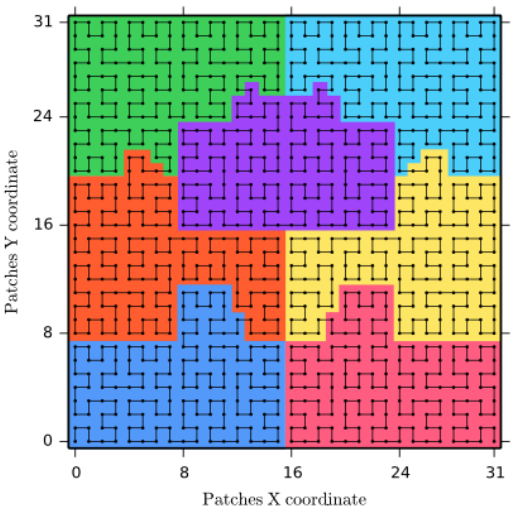
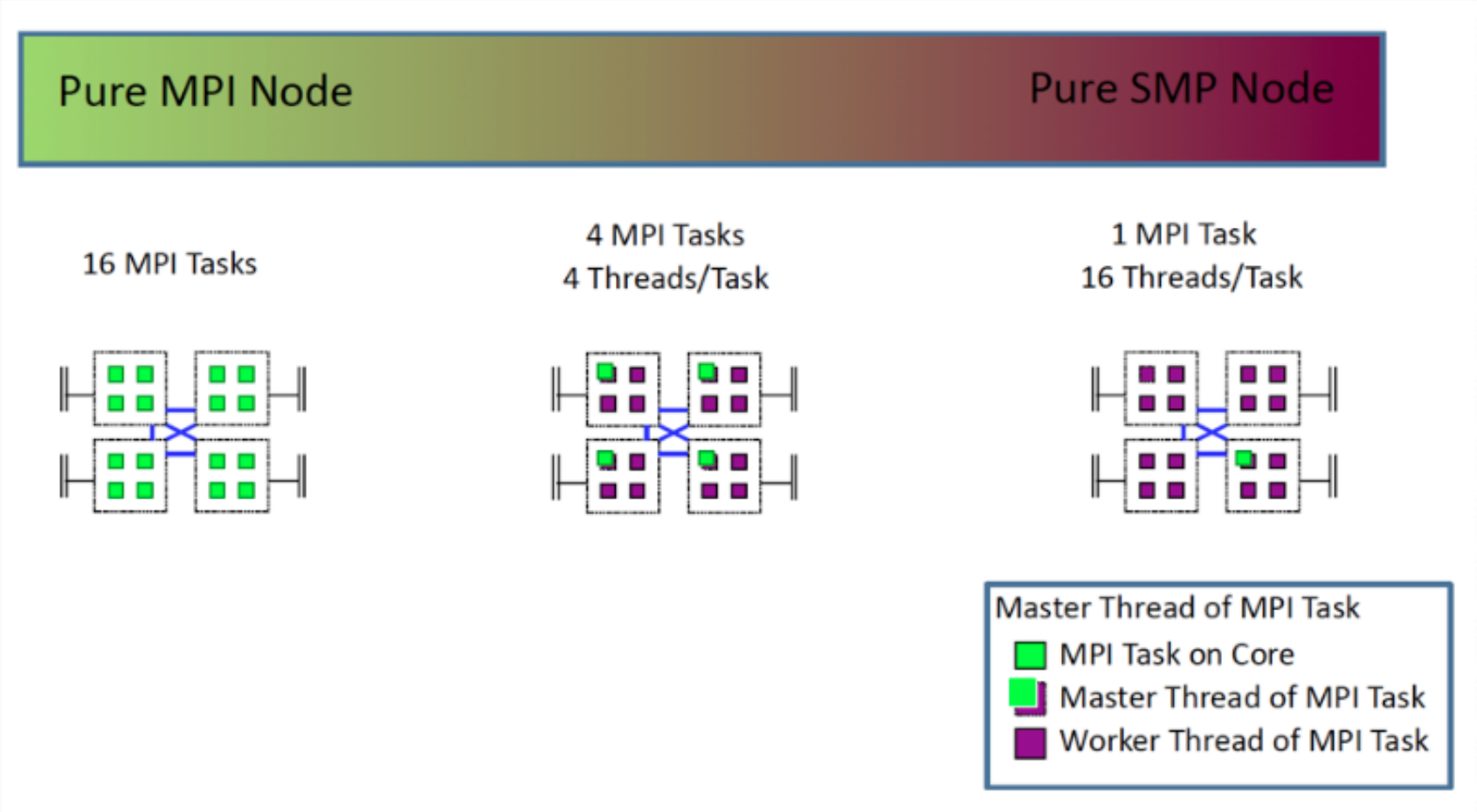


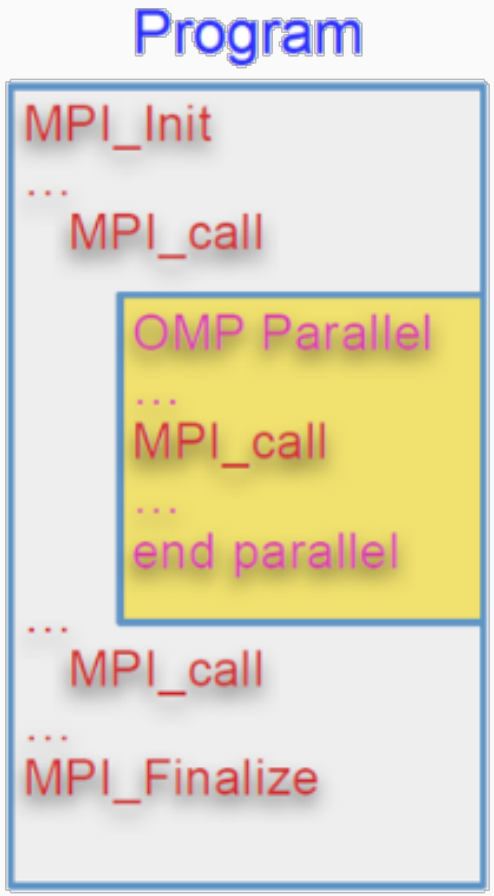
Figure 3: Example of a 32×32 patches domain decomposition, shared between 7 MPI processes. MPI domains are delimited by different colors. The Hilbert curve (black line) passes through all the patch centers (black dots). It starts from the patch with coordinates $(0, 0)$ and ends at the patch with coordinates $(31, 0)$.

Modos de operación MPI-threads



Programa híbrido

- Se inicializa MPI
- Se crea regiones paralelas OpenMP dentro de procesos MPI
- Se pueden llamar procesos MPI en regiones en paralelo y en serie
- Se finaliza MPI



Código híbrido OpenMP-MPI

```
1  #include <stdio.h>
2  #include "mpi.h"
3  #include <omp.h>
4  int main(int argc, char *argv[]) {
5  int numprocs, rank, namelen;
6  char processor_name[MPI_MAX_PROCESSOR_NAME];
7  int iam = 0, np = 1;
8  MPI_Init(&argc, &argv);
9  MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
10 MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11 MPI_Get_processor_name(processor_name, &namelen);
12 #pragma omp parallel num_threads(4) default(shared) private(iam, np)
13 {
14 np = omp_get_num_threads();
15 iam = omp_get_thread_num();
16 printf("Peekaboo desde el hilo %d de un total de %d y desde el proceso
17 %d de un total de %d en %s\n", iam, np, rank, numprocs, processor_name);
18 }
19 MPI_Finalize(); }
```

Código híbrido OpenMP-MPI: compilación

```
1  mpicc -fopenmp peekaboo_mpi_omp.c
2  mpirun -np 2 ./a.out
```

En ocasiones, el número óptimo de threads es una característica de la arquitectura de hardware, y debe ser sujeto a pruebas.

Máquinas híbridas:

Una máquina con 268 nodos Cada nodo tiene 8 núcleos
Se pueden combinar:
- 1 proceso MPI y 8 hilos OpenMP
- 2 procesos MPI y 4 hilos OpenMP
- 4 procesos MPI y 2 hilos OpenMP

2 nodos, 1 proceso MPI/nodo, 4 hilos
0 running on compute-2-25.local thread= 0 of 4
0 running on compute-2-25.local thread= 1 of 4
0 running on compute-2-25.local thread= 2 of 4
0 running on compute-2-25.local thread= 3 of 4
1 running on compute-3-14.local thread= 0 of 4
1 running on compute-3-14.local thread= 1 of 4
1 running on compute-3-14.local thread= 2 of 4
1 running on compute-3-14.local thread= 3 of 4

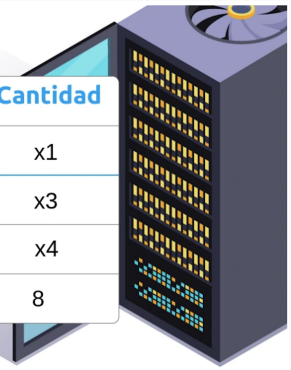
Uso en clusters: Khipu

CIENCIA DE LA COMPUTACION UTEC

KHIPU

Nodo	Cores	Mem (GB)	Storage (GB)	Cantidad
nLíder	40	128	40000	x1
nCPU*	96	390	-	x3
nGPU*	256	3192	-	x4
Total	392	3710	40000	8

*usados para procesamiento



Especificaciones	
Nombre	n004
Procesador	Intel(R) Xeon(R) Gold 6130 CPU @2.10 GHz 16 cores por socket, 32 por nodo.
Memoria	126 GB DRAM DDR4-1333 MHz
Red	Infiniband FDR MT4119

```
#!/bin/bash

# Un Job script para la ejecución de un código híbrido MPI-OpenMP

#SBATCH --job-name=hibrido_mpi_openmp
#SBATCH --output=hibrido_mpi_openmp.out
#SBATCH --ntasks=4
#SBATCH --cpus-per-task=8
#SBATCH --partition=docencia

# Cargar el modulo MPI.
module load mpich/4.0

# Configurar el valor de  OMP_NUM_THREADS con el numero de CPUs por task solicitado.
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK

# Ejecutar el proceso con mpirun. Puede notar que no es necesario especificar el flag de MPI
# '-n' puesto que automaticamente utiliza el valor de la configuración de Slurm realizada

mpirun ./hibrido

module unload mpich/4.0
```

4 procesos MPI y 8 hilos OMP por proceso

Supercomputador Perlmutter (NERSC)



Sistema heterogéneo, basado en una plataforma Cray, con 3072 CPUs (peak: 15.4 PFLOPs) y 1792 GPUs (peak: 69.5 PFLOPs). Un procesador AMD EPYC 7713 contiene 64 núcleos físicos, 128 hilos, 3.67 GHz

Perlmutter CPU

```
#!/bin/bash
#SBATCH --qos=debug
#SBATCH --time=5
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=128
#SBATCH --constraint=cpu

srun check-mpi.gnu.pm
```

Un proceso MPI por núcleo físico

Supercomputador Perlmutter (NERSC)



Perlmutter CPU

```
#!/bin/bash
#SBATCH --qos=debug
#SBATCH --time=5
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=2
#SBATCH --cpus-per-task=128
#SBATCH --constraint=cpu

export OMP_PROC_BIND=spread
export OMP_PLACES=threads
export OMP_NUM_THREADS=64

srun check-hybrid.gnu.pm
```

Un proceso MPI por socket y 1 hilo OMP por núcleo físico

Ecuacion de Poisson en 3D

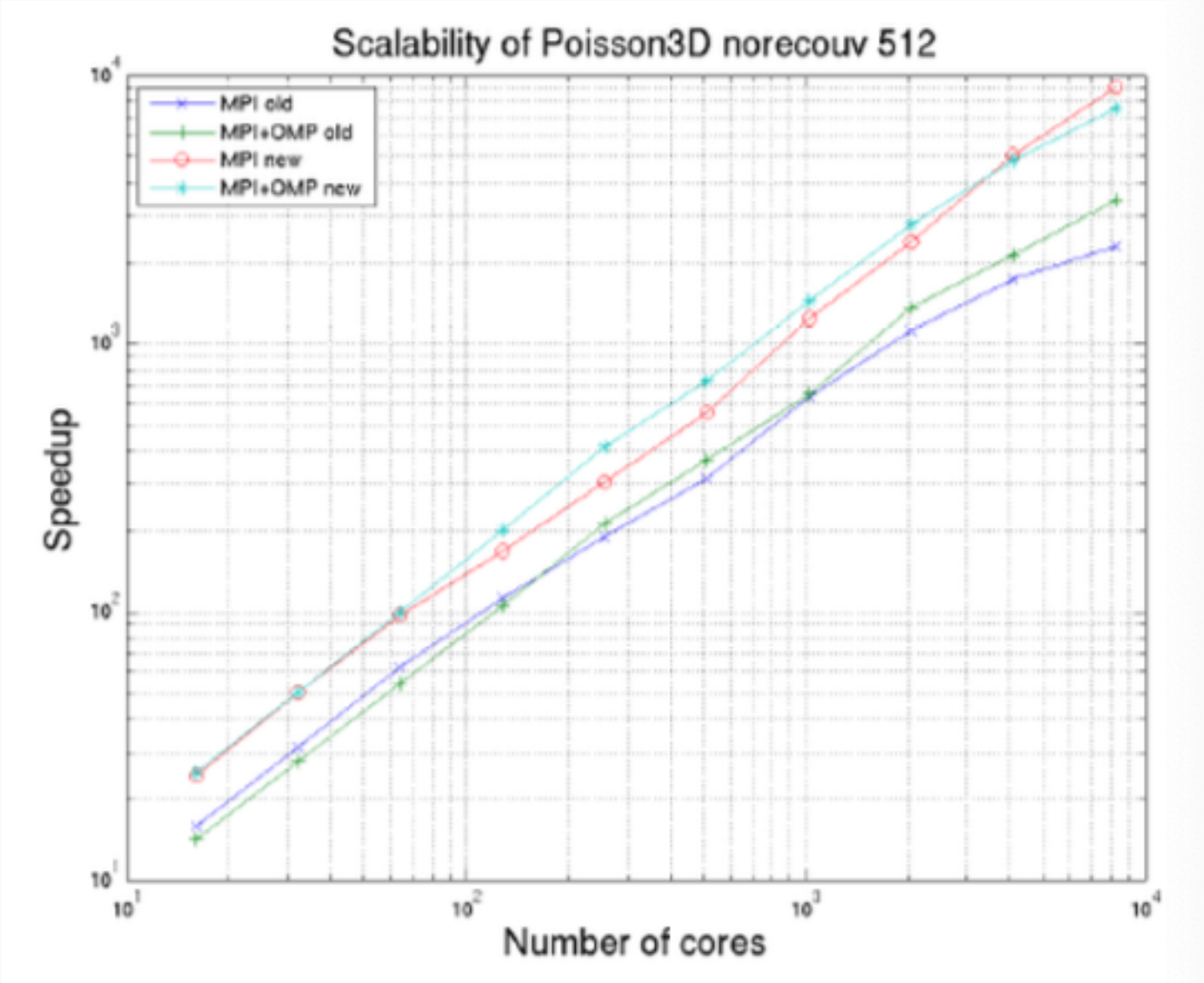
Este método resuelve la ecuación de Poisson en un dominio cúbico [0,1]x[0,1]x[0,1] usando una discretización finita y el método de Jacobi

$$\begin{cases} \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} & = f(x,y,z) \quad \text{in } [0,1]\text{x}[0,1]\text{x}[0,1] \\ u(x,y,z) & = 0. \quad \text{on the boundaries} \\ f(x,y,z) & = 2yz(y-1)(z-1) + 2xz(x-1)(z-1) + 2xy(x-1)(y-1) \\ u_{\text{exact}}(x,y) & = xyz(x-1)(y-1)(z-1) \end{cases}$$

Se discretiza en una malla regular en las tres direcciones del espacio (h=hx=hy=hz) La solución se calcula usando el método de Jacobi donde la solución de la iteración n+1 se calcula usando la inmediata adyacente iteración n

$$u_{ijk}^{n+1} = \frac{1}{6}(u_{i+1jk}^{n} + u_{i-1jk}^{n} + u_{ij+1k}^{n} + u_{ij-1k}^{n} + u_{ijk+1}^{n} + u_{ijk-1}^{n} - h^2 f_{ijk})$$

Tests realizados en un supercomputador IBM Blue Gene, de 10240 nodos con 4 núcleos cada uno



OpenMP + MPI

¿Por qué no necesariamente mas eficiente?

- OpenMP tiene **mayor escalabilidad** debido a paralelismo implícito (pero limite de hardware)
- Durante la comunicación MPI todos los **threads esperan** (idle) menos uno
- Costo excesivo en la creación de **múltiples threads**
- No existen **librerías/compiladores optimizados** en OpenMP

SPHYNX y SPHEXA

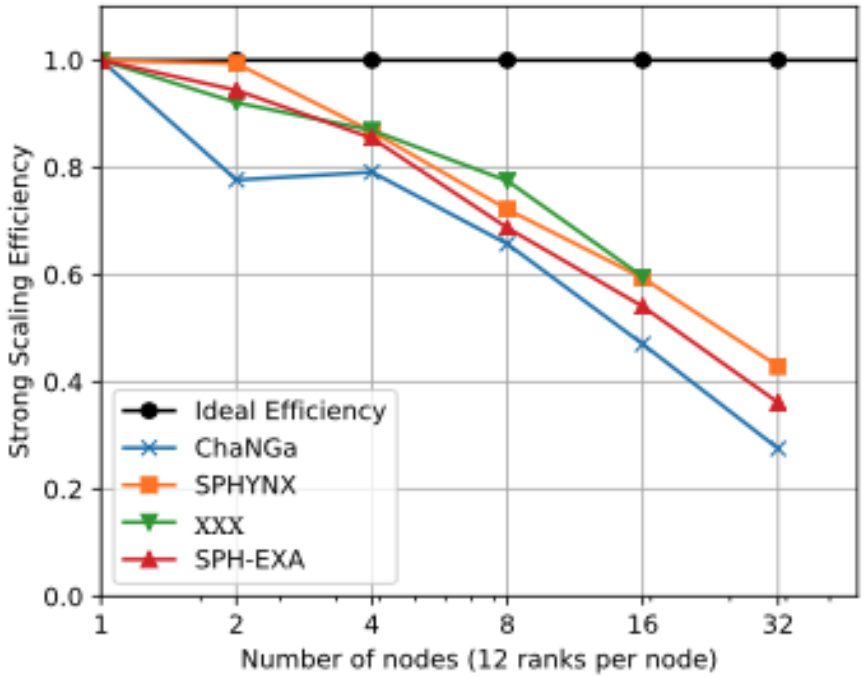


Figure 2: Parallel efficiency of ChaNGa(12 OpenMP tasks and 1 MPI rank per node), SPHYNX (12 OpenMP tasks and 1 MPI rank per node), XXX (12 MPI ranks per node) and SPH-EXA mini-app (12 MPI ranks per node) under strong scaling, in function of the number of nodes used (12 cores per node) on Piz Daint.

SPHYNX y SPHEXA

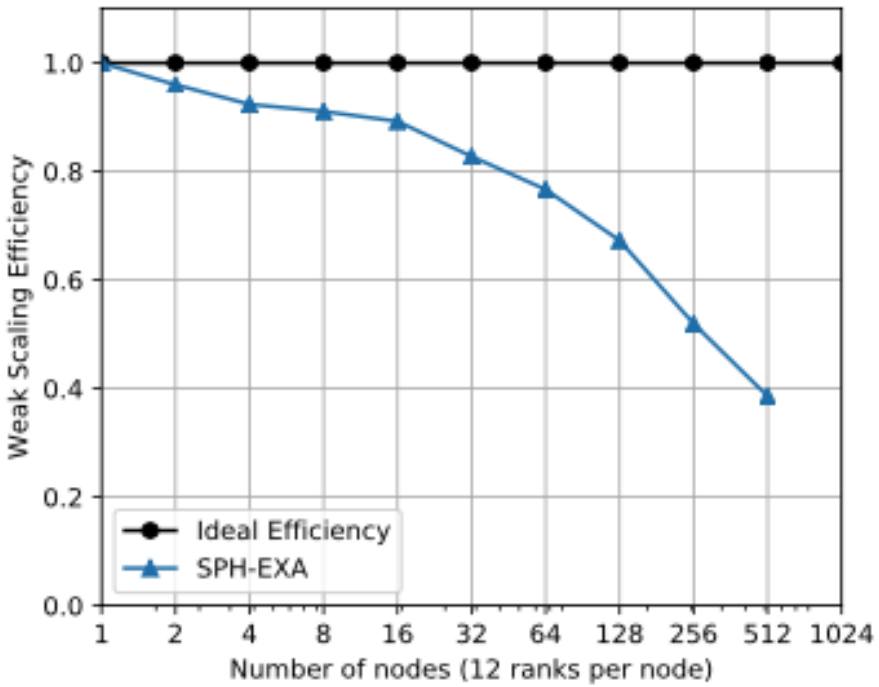


Figure 3: Parallel efficiency of the SPH-EXA mini-app (12 MPI ranks per node) under weak scaling, in function of the number of nodes used (12 cores per node) on Piz Daint.

Problema de N-cuerpos: OMP vs HPX

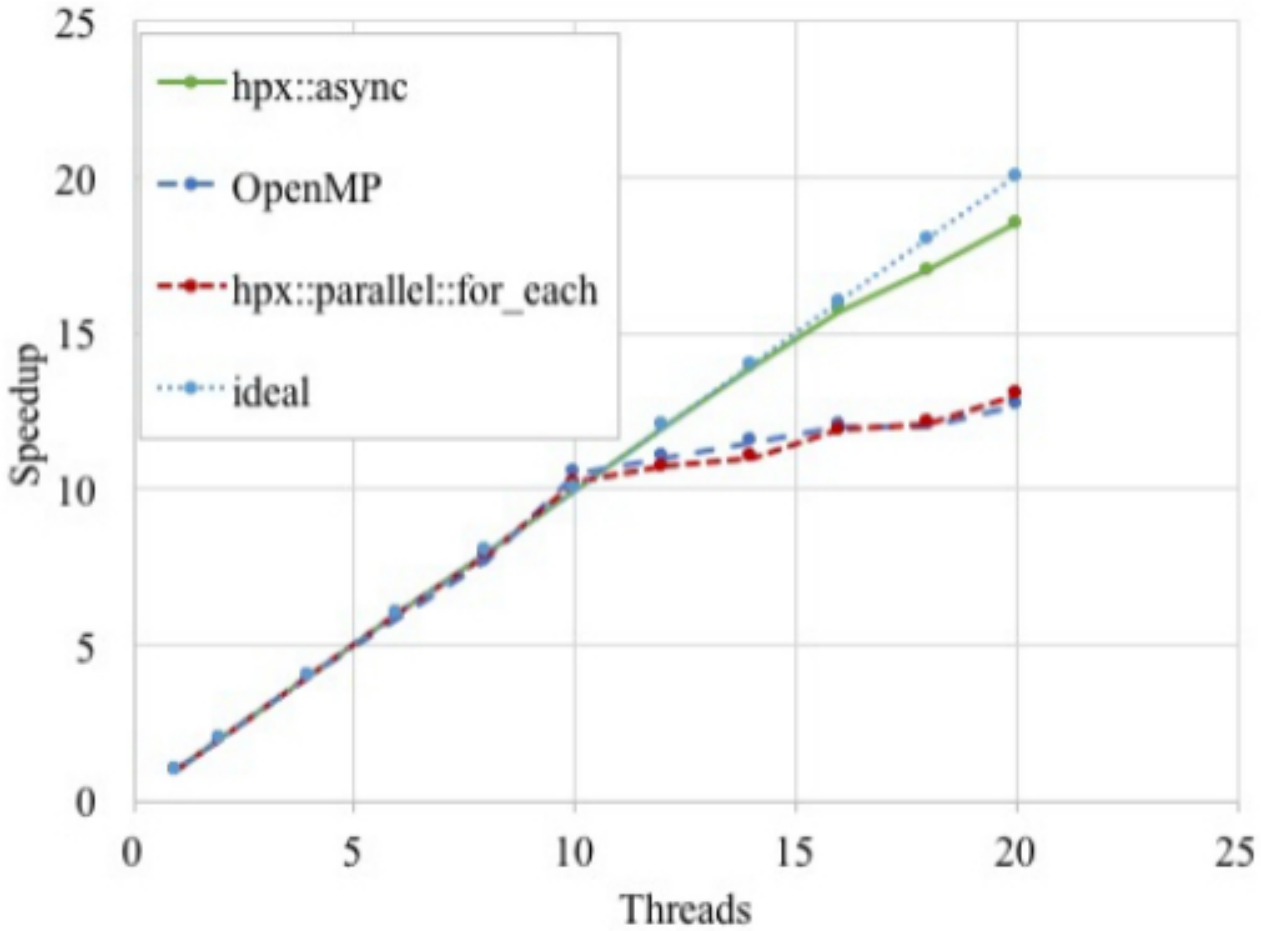


Figure 9: Comparing results of the strong scaling for the Octree construction of 10^6 particles implemented with two HPX parallelization methods and OpenMP. *hpx::async* exposes significantly better speedup, since it allows threads to continue to execute without unnecessary barriers.

Problema de N-cuerpos: Hybrid

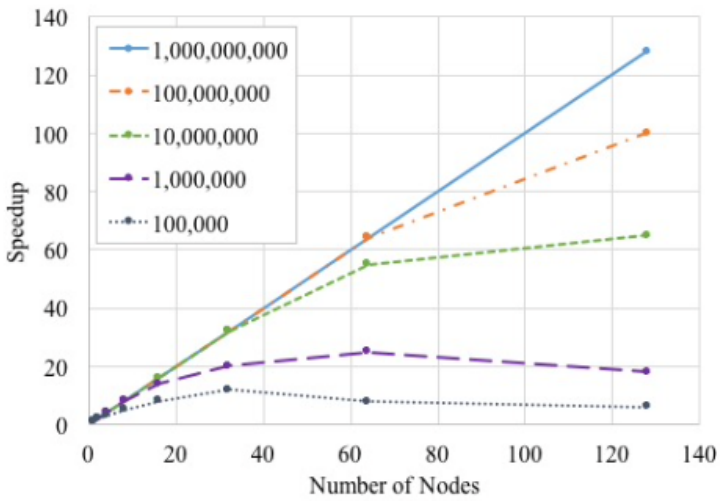


Figure 11: Strong scaling of the distributed N-Body application implemented with HPX on up to 128 nodes (20 cores per node). It shows a close to perfect HPX scalability for problem size of 10^9 particles.

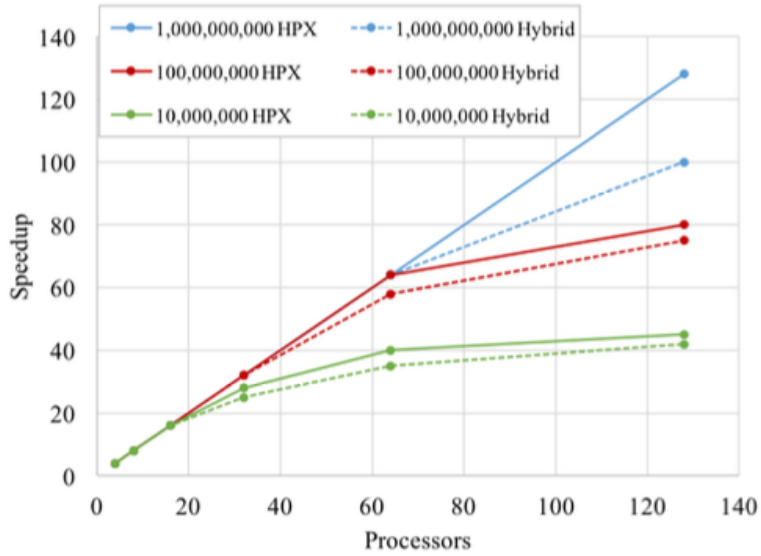


Figure 13: Comparison results of the strong scaling between HPX and a hybrid model (OpenMP/MPI), with up to 128 nodes (20 cores per node). The results illustrate a better performance for the HPX application for larger number the nodes, which is due to using a *future*-based request buffer between the remote nodes.

Problema de N-cuerpos: MPI + CUDA

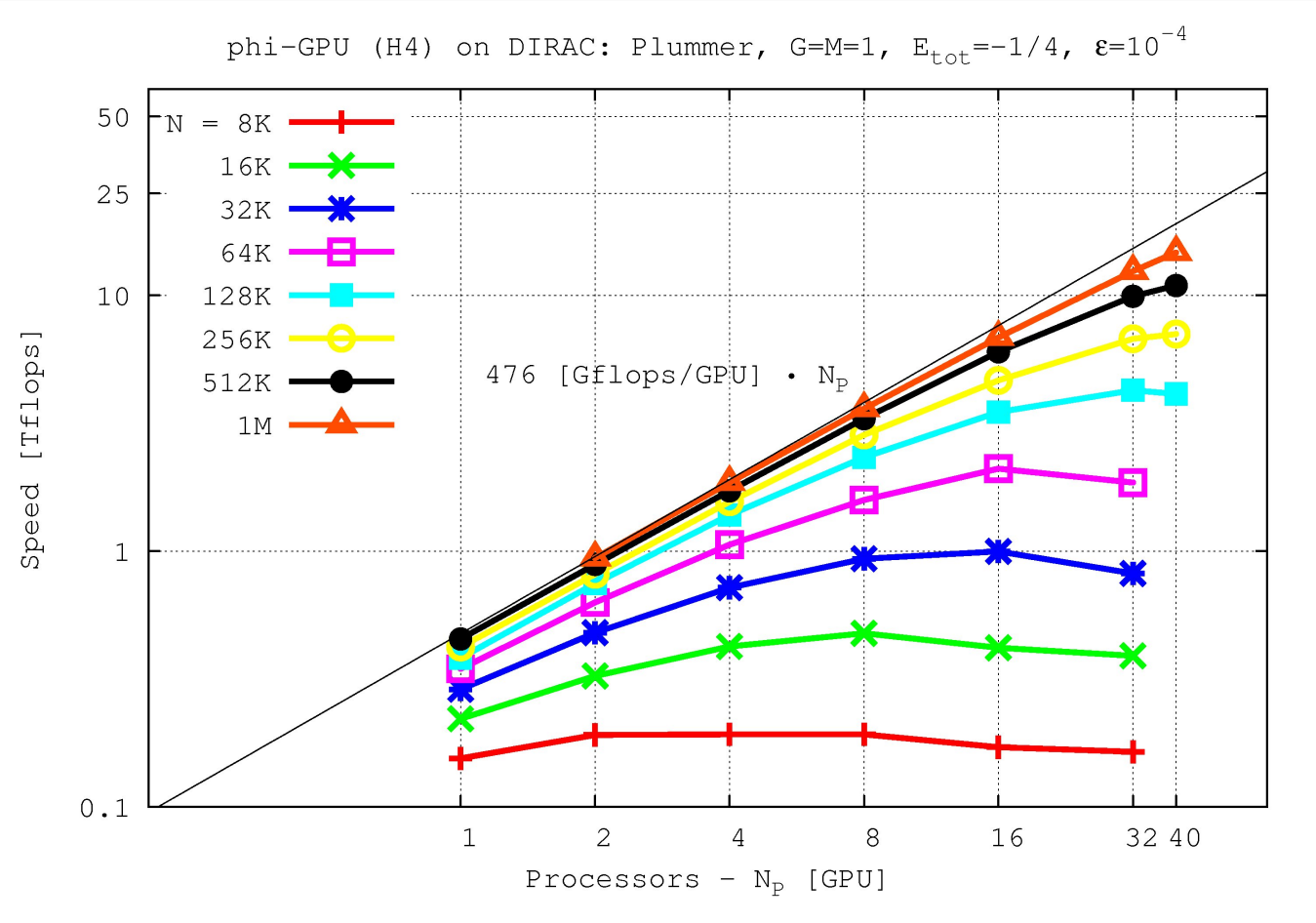


Figura 7:

Performance MPI(+CUDA) para N-cuerpos (directo)

Bibliografía i


David B. Kirk and Wen-mei W. Hwu *Programming Massively Parallel Processors: A Hands-on Approach*. 2nd. Morgan Kaufmann, 2013. isbn: 978-0-12-415992-1.


Norm Matloff. *Programming on Parallel Machines*. University of California, Davis, 2014.


Peter S. Pacheco. *An Introduction to Parallel Programming.* 1st. Morgan Kaufmann, 2011. isbn: 978-0-12-374260- 5.


Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP.* 1st. McGraw-Hill Education Group, 2003. isbn: 0071232656.


Jason Sanders and Edward Kandrot. *CUDA by Example: An Introduction to General-Purpose GPU Program- ming.* 1st. Addison-Wesley Professional, 2010. isbn: 0131387685, 9780131387683.

UTEC
UNIVERSIDAD DE INGENIERÍA
Y TECNOLOGÍA

